

7/26/23, 12:14 AM

CodeProjectFileEditFindViewToolsEducationHelpRunVirtual DesktopDebug

app.pyx tonghewaters. |

1 from kivy.app import App

2 from kivy.uix.gridlayout import GridLayout

3 from kivy.uix.label import Label

4 from kivy.uix.button import Button

5

6 class ClickButtons(App):

7 def build(self):

8 # define the window

9 self.window = GridLayout(cols=1)

10

11 # define the widgets

12

13 # add the widgets

14

15 return self.window

16

17 if __name__ == "__main__":

18 ClickButtons().run()

100% (18/23)			
#####			
xorg-libice-1.1.1	57 KB	#####	100%
ca-certificates-2023	146 KB	#####	100%
libpng-1.6.39	276 KB	#####	100%
libexpat-2.5.0	76 KB	#####	100%
kivy-2.2.1	28.5 MB	#####	100%
libwebp-base-1.3.1	391 KB	#####	100%
libzfp-0.11.1	188 KB	#####	100%
pip-23.2.1	1.3 MB	#####	100%
xz-5.2.6	469 KB	#####	100%
nom-3.5.0	2.7 MB	#####	100%
docutils-0.20.1	784 KB	#####	100%
gettext-0.21.1	4.1 MB	#####	100%
lib-2.16.4	471 KB	#####	100%
libgomp-13.1.0	469 KB	#####	100%
xorg-renderproto-0.1	9 KB	#####	100%
libxcb-1.15	375 KB	#####	100%
pysocks-1.7.1	19 KB	#####	100%
pillow-10.0.0	44.5 MB	#####	100%
libsystemd-253	372 KB	#####	100%
libgcc-ng-13.1.0	758 KB	#####	100%
lcms2-2.15	236 KB	#####	100%
lib_samlinux-64-2.4	688 KB	#####	100%
libdb-6.2.32	23.3 MB	#####	100%
freetype-2.12.1	611 KB	#####	100%
python-3.10.12	24.4 MB	#####	100%
zstd-1.5.2	421 KB	#####	100%
libcap-2.67	37 KB	#####	100%

Code - Packaging GUI Applications

Guide x

Collapse

GUI Applications

The Kivy Library

You can create graphical user interface (GUI) applications with Python. There are a variety of tools to do this, but we are going to focus on the Kivy library. Kivy allows you to easily create cross-platform applications. Take a look at the gallery to see kinds of projects you can create with Kivy.

Another benefit to using Kivy is that it works with Pyinstaller to create an executable file. To be fair, Pyinstaller also works with other Python GUI libraries like PyQt and Tkinter.

Let's start by creating the `gui` virtual environment with Conda. Enter the following command in the terminal (bottom-left).

```
conda create --name gui -y
```

Once Conda finishes creating the virtual environment, activate it. You should see `(gui)` appear at the beginning of the prompt.

```
conda activate gui
```

Next install the Kivy library. This is not found in the Anaconda repository, so you need to specify the `conda-forge` channel when installing.

```
conda install -c conda-forge kivy -y
```

Application Basics

We are not going to build a sophisticated application with Kivy. In fact, the application is quite boring. However, the idea behind this assignment is to see how Pyinstaller creates GUI executables. With that in mind, let's build this very simple application that has a counter. Press the button with a `+` to increase the counter by one, and press the `-` button to decrease the counter.

Start by importing the various modules we need. Our Kivy application is going to be a subclass of the `App` class. We are going to use a simple grid layout, which means our application is divided into columns and rows. Widgets are placed in the application according to the layout. A `Label` is the widget used for displaying text, and a `Button` widget is used for interaction.

```
from kivy.app import App
from kivy.uix.gridlayout import GridLayout
from kivy.uix.label import Label
from kivy.uix.button import Button
```

Create the `ClickButtons` class (which will be the name of our application), which inherits from `App`. Create the `build()` method and define `self.window` as a grid layout with a single column. That means each widget will be stacked on top of one another. The last line of the `build` method should return `self.window`. Finally, create a conditional (outside of the class) that checks if the script is being run directly. If so, call the `run()` method. The comments are there to help us organize our code, they are not required.

```
from kivy.app import App
from kivy.uix.gridlayout import GridLayout
from kivy.uix.label import Label
from kivy.uix.button import Button

class ClickButtons(App):
    def build(self):
        # define the window
        self.window = GridLayout(cols=1)

        # define the widgets

        # add the widgets

        return self.window

if __name__ == "__main__":
    ClickButtons().run()
```

Believe it or not, that is enough code to generate a GUI. It does not do anything because it is a blank window, but that is all it takes to get started with Kivy. Run the following command in the terminal.

```
python app.py
```

Kivy will return lots of text as it prepares the GUI. If there is a problem with your code, Kivy will return an error message (which are very user friendly when compared to Python) and show you the prompt. If you do not see the prompt (looks something like `-/workspace/app$`), that means your application is running. Click the link below to preview your work.

Preview the GUI

You should see a new tab appear in the top-left panel. In it, you will see our application. There is a window entitled `ClickButtons`. Our window is hard to see. For starters, it is bigger than the panel and it is a black window on a black background. We will address these issues on the next page. It is important to close the GUI window. If not, Python will keep running the window even though we do not see it. After closing the window, look at the terminal; you should see the prompt. That is how we know our GUI application has stopped running. You can now close the tab that contained the GUI application.

Try this variation:

- Change the text that appears in the title bar of the Kivy app to `My_First_Kivy_App`.

```
class My_First_Kivy_App(App):
    def build(self):
        # define the window
        self.window = GridLayout(cols=1)

        # define the widgets

        # add the widgets

        return self.window

if __name__ == "__main__":
    My_First_Kivy_App().run()
```

Run the application again and verify that you see the title change in the window. Be sure to change the code back to `ClickButtons` as that will be used in the rest of the code examples.

```
python app.py
```

Preview the GUI

Deactivate the `gui` virtual environment.

```
conda deactivate
```

▼ Code

Your code should look like this:

```
from kivy.app import App
from kivy.uix.gridlayout import GridLayout
from kivy.uix.label import Label
from kivy.uix.button import Button

class ClickButtons(App):
    def build(self):
        # define the window
        self.window = GridLayout(cols=1)

        # define the widgets

        # add the widgets

        return self.window

if __name__ == "__main__":
    ClickButtons().run()
```

Reading Question

Fill in the blanks below.

- Kivy apps must inherit from the `App` class.
- The class for the Kivy app must contain the `build` method.
- The aforementioned method must return the layout.

The correct answers are:

- Kivy apps must inherit from the **App** class.
- The class for the Kivy app must contain the **build** method.
- The aforementioned method must return **the layout**.

Kivy applications inherit from the `App` class and must contain the `build` method. This method returns the layout. In our example, it is a grid layout called `self.window`.

Check It

Next